

# 如何写一个ROS2 Controller

## 前言

尽量多看Meta-ROS(

[https://github.com/Meta-Team/Meta-ROS/tree/master/decomposition/meta\\_chassis\\_controller](https://github.com/Meta-Team/Meta-ROS/tree/master/decomposition/meta_chassis_controller),

[https://github.com/Meta-Team/Meta-ROS/tree/master/decomposition/meta\\_gimbal\\_controller](https://github.com/Meta-Team/Meta-ROS/tree/master/decomposition/meta_gimbal_controller),

[https://github.com/Meta-Team/Meta-ROS/tree/master/decomposition/meta\\_shoot\\_controller](https://github.com/Meta-Team/Meta-ROS/tree/master/decomposition/meta_shoot_controller)

) 或者ROS2官方Controller实现的源码([https://github.com/ros-controls/ros2\\_controllers](https://github.com/ros-controls/ros2_controllers)),多思考

可以先基于现有的controller修改

## 环境配置(Plugin lib注册Controller, 以及C++读取YAML里面的parameters)

ros2编译使用ament\_cmake管理

## Plugin lib注册Controller

Controller都是通过Plugin\_lib注册的.

关于Plugin\_lib注册,你需要三件事情:

1. 在CMakeLists.txt 里面添加注册的XML描述文件

```
35 pluginlib_export_plugin_description_file(  
36     controller_interface meta_gimbal_controller.xml)  
37
```

2. XML中type对应C++里面的类名,name 对应parameter XML的句柄(字符串)就行, base\_class\_type 填继承的基类

```

1 <library path="meta_gimbal_controller">
2   <class name="meta_gimbal_controller/GimbalController"
3     type="gimbal_controller::GimbalController"
4     base_class_type="controller_interface::ChainableControllerInterface">
5   <description>
6     GimbalController ros2_control controller.
7   </description>
8 </class>
9   <class name="meta_gimbal_controller/GimbalPositionController"
10     type="gimbal_controller::GimbalPositionController"
11     base_class_type="controller_interface::ChainableControllerInterface">
12   <description>
13     GimbalController ros2_control controller.
14   </description>
15 </class>
16 </library>

```

### 3. C++类名对应以及C++头文件内注册

```

25 namespace gimbal_controller {
26   // name constants for state interfaces
27   static constexpr size_t STATE_MY_ITFS = 0;
28
29   // name constants for command interfaces
30   static constexpr size_t CMD_MY_ITFS = 0;
31
32   class GimbalPositionController : public controller_interface::ChainableControllerInterface {
33   public:
34     GimbalPositionController() = default;
35
36     controller_interface::CallbackReturn on_init() override;
37
38     controller_interface::InterfaceConfiguration
39     command_interface_configuration() const override;
40
41     controller_interface::InterfaceConfiguration
42     state_interface_configuration() const override;
43
44     controller_interface::CallbackReturn

```

头文件

注意首先要 `#include "pluginlib/class_list_macros.hpp"`, 以及需要把 `namespace(gimbal_controller)` 加在类名前

```

282
283 } // namespace gimbal_controller
284
285 #include "pluginlib/class_list_macros.hpp"
286 ✦
287 PLUGINLIB_EXPORT_CLASS(gimbal_controller::GimbalPositionController,
288                       controller_interface::ChainableControllerInterface)

```

源文件末尾

## 读取YAML里面的parameters

在C++侧, 通过param\_listener来读取params.

也是三个步骤

1. 创建param\_listener的描述YAML文件, 需要对应真正的parameter结构

比如要读取的parameter长这样:

```

80 gimbal_position_controller:
81   ros__parameters:
82     yaw_gimbal_joint:
83       enable: true
84       name: yaw_gimbal_joint
85
86     pitch_gimbal_joint:
87       enable: true
88       name: pitch_gimbal_joint
89
90     imu_sensor: dm_imu
91     gains:
92       yaw_gimbal_joint_pos2vel:
93         { p: 20.0, i: 0.0, d: 0.2, i_clamp_max: 1.0, i_clamp_min: -1.0, antiwindup: true }
94

```

Meta-ROS/meta\_bringup/config/hero.yaml

则创建这样的. 开头的 `gimbal_position_controller`, 其实可以随便填, 这个是param\_listener的生成名字, 之后在C++文件里面使用时需要对应(头文件名和类名和这个有关)

```

1 gimbal_position_controller:
2   yaw_gimbal_joint:
3     enable:
4       {
5         type: bool,
6         default_value: true,
7         description: "Specifies whether the yaw gimbal joint is enabled.",
8       }
9     name:
10      {
11        type: string,
12        default_value: "yaw_gimbal_joint",
13        description: "Specifies joint of the yaw gimbal.",
14      }
15   pitch_gimbal_joint:
16     enable:
17       {
18         type: bool,
19         default_value: true,
20         description: "Specifies whether the pitch gimbal joint is enabled.",
21       }
22     name:
23       {
24         type: string,
25         default_value: "pitch_gimbal_joint",
26         description: "Specifies joint of the pitch gimbal.",
27       }
28   imu_sensor:
29     {
30       type: string,
31       default_value: "dm_imu",
32       description: "the sensor interface name of the imu",
33     }
34

```

Meta-ROS/decomposition/meta\_gimbal\_controller/src/meta\_gimbal\_position\_controller.yaml

2. 然后在对应的CMakeLists.txt( `Meta-ROS/decomposition/meta_gimbal_controller/CMakeLists.txt` )文件注册这个YAML文件, 用于生成library, 并链接生成出来的parameters库

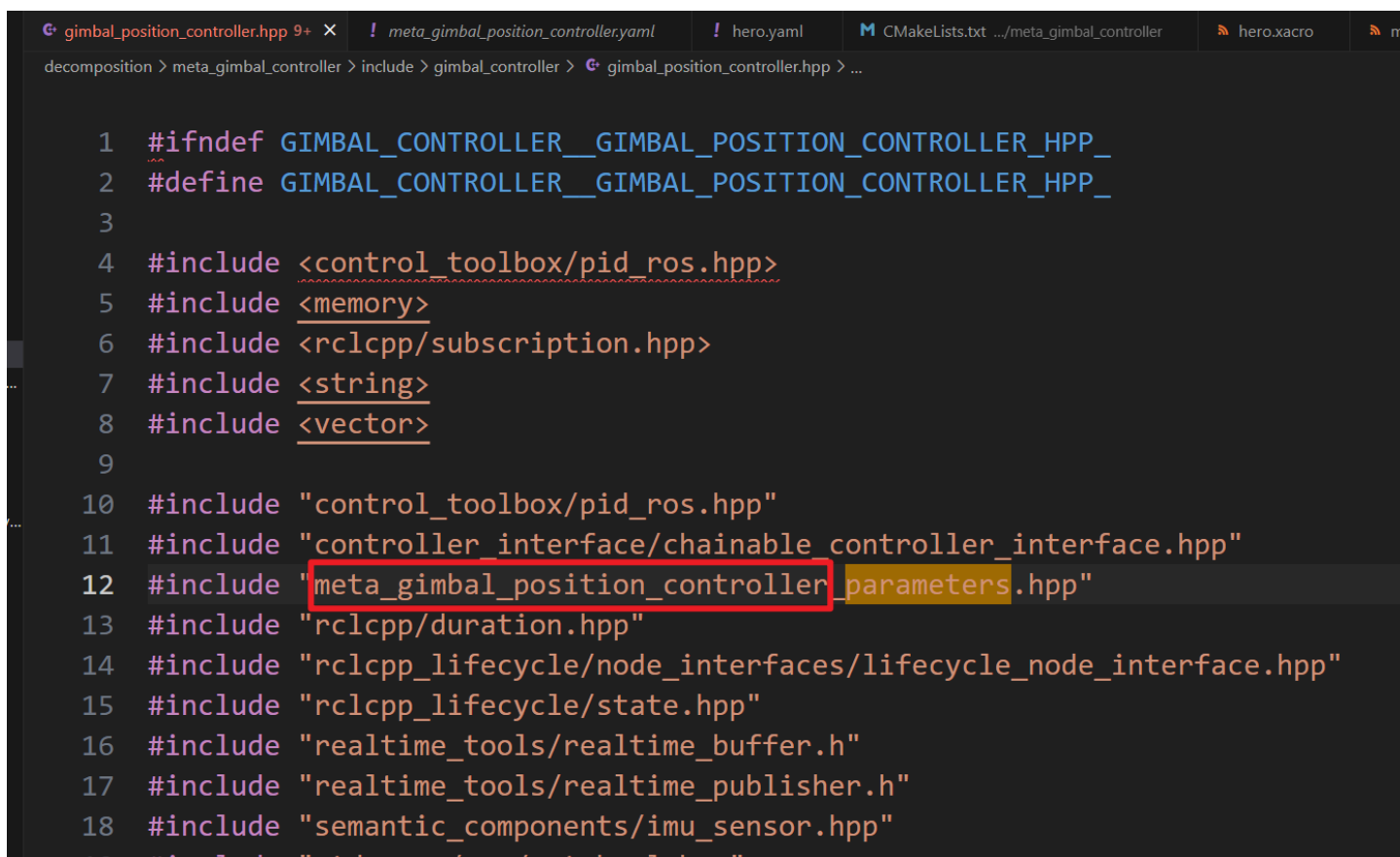
```
generate_parameter_library(
  meta_gimbal_position_controller_parameters
  src/meta_gimbal_position_controller.yaml
)
target_link_libraries(
  ${PROJECT_NAME}
  ${PROJECT_NAME}_parameters
  meta_gimbal_position_controller_parameters
)
```



这里`${PROJECT_NAME}_parameters`是另一个gimbal\_controller的param\_listener

### 3. 在C++文件里读取

首先在Controller头文件里, 添加对应名字(yaml里面第一行名称)的头文件



```
1  #ifndef GIMBAL_CONTROLLER__GIMBAL_POSITION_CONTROLLER_HPP_
2  #define GIMBAL_CONTROLLER__GIMBAL_POSITION_CONTROLLER_HPP_
3
4  #include <control_toolbox/pid_ros.hpp>
5  #include <memory>
6  #include <rclcpp/subscription.hpp>
7  #include <string>
8  #include <vector>
9
10 #include "control_toolbox/pid_ros.hpp"
11 #include "controller_interface/chainable_controller_interface.hpp"
12 #include "meta_gimbal_position_controller_parameters.hpp"
13 #include "rclcpp/duration.hpp"
14 #include "rclcpp_lifecycle/node_interfaces/lifecycle_node_interface.hpp"
15 #include "rclcpp_lifecycle/state.hpp"
16 #include "realtime_tools/realtime_buffer.h"
17 #include "realtime_tools/realtime_publisher.h"
18 #include "semantic_components/imu_sensor.hpp"
```

然后在类定义两个成员, 注意名称一致(这里还是 `gimbal_position_controller`):

```
protected:
  std::shared_ptr<gimbal_position_controller::ParamListener> param_listener_;
  gimbal_position_controller::Params params_;
```

示例读取: Meta-

ROS/decomposition/meta\_gimbal\_controller/src/gimbal\_position\_controller  
.cpp

在controller的 `on_configure()` 方法里面先调用 `get_params()` , 然后就可以正常用了  
比如想获取 `pitch_gimbal_joint.name` , 则是 `params_.pitch_gimbal_joint.name`

```
52     controller_interface::CallbackReturn
53     GimbalPositionController::on_configure(const rclcpp_lifecycle::State& /*previous_state*/)
54     {
55         params_ = param_listener_->get_params();
56         // Initialize PIDs
57         yaw_pos2vel_pid_ = std::make_shared<control_toolbox::PidROS>(
58             get_node(), "gains." + params_.yaw_gimbal_joint.name + "_pos2vel", true);
59     }
```

## 编写 ROS2 Control 控制器

本教程将指导您如何基于提供的模板代码编写一个 ROS2 Control 控制器。这个控制器继承自 `ChainableControllerInterface` , 支持控制器串联模式。

<https://l5595ex0wg.feishu.cn/docx/DKyfddYjQoDqqEx3TFTcprOGn3b>

代码块

```
1  #include "my_controller/my_base_controller.hpp"
2
3  #include <string>
4
5  #include "rclcpp/logging.hpp"
6  #include "hardware_interface/loaned_command_interface.hpp"
7  #include "hardware_interface/loaned_state_interface.hpp"
8  // some commands
9  // first time run
10 // ./tools/install.sh
11 // 1. setup can
12 // ./tools/setupcan.sh
13 // 2. check can device number connected PC connection
14 // ip adress
15 // or ip a for short
16 // 3 . check RS485 and UART connection
17 // ls /dev/tty_*
18 namespace my_controller
19 {
20     my_controller::my_controller() {}
21
22     controller_interface::CallbackReturn my_controller::on_init()
23     {
24         try
25         {
```

```

26         param_listener_ = std::make_shared<my_controller::ParamListener>
(get_node());
27     } catch (const std::exception &e) {
28         std::cerr << "Exception thrown during controller's init with
message: "
29                 << e.what() << std::endl;
30         return controller_interface::CallbackReturn::ERROR;
31     }
32
33     return controller_interface::CallbackReturn::SUCCESS;
34 }
35
36     controller_interface::CallbackReturn my_controller::on_configure(const
rclcpp_lifecycle::State & previous_state)
37     {
38         params_ = param_listener_->get_params();
39
40         // To be implemented
41         // 0. param listener init
42         // TODO: 1. init DJI vel2eff PID tool;
43         // 2.-1 topics QoS init, ready for subscription
44         // 2.init reference subscriber(input_ref_ (using a customized msg
including 2 double, or std_msg) and its callback)
45         // 2.5 create a new interface(msg), follow guide(Meta-
ROS/interfaces/xxx_interface/msg/yyy.msg, remember to register it in
CMakeLists.txt), or using std_msgs/msg/Float64MultiArray (ls
/opt/ros/humble/include/std_msgs/std_msgs/msg/float64_multi_array.hpp)
46         // 2.6 for a subscriber, you need to have a
rclcpp::Subscription<T>::SharedPtr in class,
47         // plus a realtime_tools::RealtimeBuffer<std::shared_ptr<T>> buffer in
class, which you update in a class member function callback.
48         // 2.7 set realtime buffer's content to NaN(i.e. reset) before the
first valid bunch of msg's arrival
49         // 2.8 add update_reference_from_subscribers function to class, copy
realtime buffer's content to reference_interfaces[xxx]
50         // 3. state publisher(1dof():DJI PID), first create a normal
s_publisher_, then use s_publisher_ to create a REALTIME publisher
state_publisher_
51         // 4. (optional) RCLCPP_INFO
52
53         RCLCPP_INFO(get_node()->get_logger(), "configure successful");
54         return controller_interface::CallbackReturn::SUCCESS;
55     }
56     // TODO on_set_chained_mode()
57     controller_interface::InterfaceConfiguration
my_controller::command_interface_configuration() const
58     {

```

```

59     controller_interface::InterfaceConfiguration config;
60     config.type =
controller_interface::interface_configuration_type::INDIVIDUAL;
61
62     // To be implemented
63     // TODO
64     // 0. create controller_interface::InterfaceConfiguration
65     // 1. dji command intf, 2. unitree command intf
66     // get name of joints using params_.xxx
67     // 3. return config
68     return config;
69 }
70
71     controller_interface::InterfaceConfiguration
my_controller::state_interface_configuration() const
72     {
73         controller_interface::InterfaceConfiguration config;
74         config.type =
controller_interface::interface_configuration_type::INDIVIDUAL;
75
76         // To be implemented
77         // similar to controller intf
78         // only need DJI's velocity state, not unitree's velocity state
79         return config;
80     }
81
82     controller_interface::CallbackReturn
my_controller::on_activate(const rclcpp_lifecycle::State &
/*previous_state*/) {
83
84         // To be implemented
85         // Set default value(reset, NaN) in command
86         // cf. gimbal_position_controller's implementation
87         return controller_interface::CallbackReturn::SUCCESS;
88     }
89
90     controller_interface::CallbackReturn
my_controller::on_deactivate(const rclcpp_lifecycle::State &
/*previous_state*/) {
91
92         // re
93         return controller_interface::CallbackReturn::SUCCESS;
94     }
95
96     // TODO update_reference_from_subscribers()
97
98     controller_interface::return_type
my_controller::update_and_write_commands(const rclcpp::Time & time, const
rclcpp::Duration & period)

```

```

99     {
100         // Overall: check whether or not,
            input(reference(reference_interfaces), state(realtime buffer, or
            state_interfaces)) are valid(not NAN), then first compute commands(if
            necessary) and then send them respectively
101         // finally, use state_publisher_ to publish PID state(error, feedback,
            reference,time, output)
102         reference_interfaces_[1];
103         return controller_interface::return_type::OK;
104     }
105
106
107 }
108
109 #include "pluginlib/class_list_macros.hpp"
110
111 PLUGINLIB_EXPORT_CLASS(
112     my_controller::my_controller,
113     controller_interface::ChainableControllerInterface)

```

## 控制器生命周期概述

ROS2 Control 控制器遵循特定的生命周期，以下是主要方法及其作用：

### 1. 初始化 (on\_init)

cpp

controller\_interface::CallbackReturn my\_controller::on\_init()

- 初始化控制器参数监听器
- 处理初始化过程中的异常
- 返回 `SUCCESS` 或 `ERROR` 表示初始化结果

### 2. 配置 (on\_configure)

cpp

controller\_interface::CallbackReturn my\_controller::on\_configure(const rclcpp\_lifecycle::State & previous\_state)

- 获取参数配置
- 初始化 PID 控制器（如 DJI vel2eff 工具）
- 设置 QoS 配置，准备订阅
- 创建参考输入订阅器和回调函数
- 创建状态发布者

- 添加调试信息日志

### 3. 命令接口配置 (command\_interface\_configuration)

cpp

```
controller_interface::InterfaceConfiguration my_controller::command_interface_configuration()  
const
```

- 定义控制器需要的命令接口
- 设置接口类型为 `INDIVIDUAL` (单独接口)
- 添加 DJI 和 Unitree 等硬件的命令接口
- 使用参数中的关节名称配置接口

### 4. 状态接口配置 (state\_interface\_configuration)

cpp

```
controller_interface::InterfaceConfiguration my_controller::state_interface_configuration() const
```

- 定义控制器需要的状态接口
- 设置接口类型为 `INDIVIDUAL`
- 添加 DJI 速度状态接口 (不需要 Unitree 速度状态)

### 5. 激活与停用 (on\_activate/on\_deactivate)

cpp

```
controller_interface::CallbackReturn my_controller::on_activate(const rclcpp_lifecycle::State &  
/*previous_state*/)
```

```
controller_interface::CallbackReturn my_controller::on_deactivate(const rclcpp_lifecycle::State  
& /*previous_state*/)
```

- 激活时重置命令值为默认值或 NaN
- 停用时进行资源清理

### 6. 更新与写入命令 (update\_and\_write\_commands)

cpp

```
controller_interface::return_type my_controller::update_and_write_commands(const  
rclcpp::Time & time, const rclcpp::Duration & period)
```

- 检查输入 (参考接口、状态接口) 是否有效 (非 NaN)
- 计算命令 (如需要)
- 发送命令到硬件接口
- 使用状态发布者发布 PID 状态 (错误、反馈、参考、时间、输出)

# 实现步骤指南

## 7. 创建自定义消息接口

如果需要自定义消息类型：

- 在 `xxx_interface/msg/` 目录下创建 `.msg` 文件
- 在 `CMakeLists.txt` 中注册消息
- 或者使用标准消息类型如 `std_msgs/msg/Float64MultiArray`

## 8. 实现订阅器

在类中添加：

cpp

```
rclcpp::Subscription<T>::SharedPtr subscription_;
```

```
realtime_tools::RealtimeBuffer<std::shared_ptr<T>> buffer_;
```

- 实现回调函数更新实时缓冲区
- 在配置阶段初始化订阅器
- 在第一个有效消息到达前将实时缓冲区内容重置为 NaN

## 9. 实现参考更新

添加 `update_reference_from_subscribers()` 方法：

- 从实时缓冲区复制内容到 `reference_interfaces_[xxx]`
- 处理数据有效性检查

## 10. 实现状态发布者

- 创建普通发布者 `s_publisher_`
- 使用普通发布者创建实时发布者 `state_publisher_`
- 在更新方法中发布 PID 状态信息

## 11. 处理控制器串联模式

实现 `on_set_chained_mode()` 方法（如需要）：

- 处理控制器在串联模式下的特殊行为

# 编译与部署

## 1. 编译控制器：

bash

```
colcon build --packages-select my_controller
```

## 2. 设置 CAN 总线（如需要）：

```
bash
```

```
./tools/setupcan.sh
```

## 3. 检查设备连接：

```
bash
```

```
ip a # 检查网络设备ls /dev/tty_* # 检查串口设备
```

## 调试提示

- 使用 `RCLCPP_INFO` 添加日志输出
- 确保所有接口名称与硬件配置匹配
- 验证参数配置正确加载
- 检查实时缓冲区是否正确更新

通过遵循以上步骤，您可以实现一个完整的 ROS2 Control 控制器，能够与硬件接口交互并实现所需的控制逻辑。

- 关于配置VSCode IntelliSense 代码提示：去[https://github.com/ErickKramer/ros2\\_with\\_vscode](https://github.com/ErickKramer/ros2_with_vscode) 下载.vscode文件夹，放置到工作目录(Meta-ROS)下，再用VSCode打开Meta-ROS目录即可拥有完整的代码提示功能。注：需要build之后才能正常工作，如果出问题可以尝试重启VSCode。